[정규강의]

융합_데이터분석 기술을 활용한 디지털콘텐츠 & 마케팅 기획

[DATA] 데이터분석 도구 Python 학습

01. 데이터분석을 위한 Python 첫걸음

Python을 배우는 목적은
코드를 외우는 것이 아니라
데이터를 이용해 문제를
해결하는 방법을 배우는 것


STARBUCKS
YOUR FAVORITES
YOUR WAY.
Download the app to customize your drink, order ahead and earn Rewards.
OUR STORE HOURS
Monday 6:00 AM - 8:00 PM
Tuesday 6:00 AM - 8:00 PM
Wednesday 6:00 AM - 8:00 PM
Thursday 6:00 AM - 8:00 PM
Friday 6:00 AM - 9:00 PM
Saturday 6:00 AM - 9:00 PM
Sunday 6:30 AM - 8:00 PM
We're happy to provide braille and large-print menus on request.
Aira Access location
WE'RE HIRING
Be part of something bigger.
FOR OUR PROTECTION
1-877-728-9349
130

스타벅스가 **다음 달 신메뉴를**
출시하려고 합니다.
무엇을 근거로 결정할까요?

- 어떤 메뉴가 많이 팔렸는가?
- 어떤 연령대가 구매했는가?
- 어느 시간대에 판매량이 높은가?
- SNS에서 어떤 메뉴가 많이 언급되는가?
- 날씨와 판매량은 관계가 있는가?

이것들은 결국 모두
Data Problem입니다

01. 우리는 왜 Python을 배우는가?

1) Data = 데이터, 자료

- 사전적 의미 : 관찰·측정·조사 등을 통해 얻어진 사실이나 값

날짜	메뉴	가격	판매량
08/20	아메리카노	4,500	128
08/20	카페라떼	5,000	83
08/20	콜드브루	5,500	61

- 이 데이터를 사람이 10행 정도 보는 것은 쉽습니다
- 하지만 100만 행이라면?
- → 컴퓨터에게 일을 시켜야 합니다.

02. 데이터 분석 과정에서 Python은 어디에 있는가?

1) Data Analysis Process

- Data Analysis = 데이터 분석
- 데이터를 수집·정제·변환·분석하여 의미 있는 정보와 의사결정 근거를 찾아내는 과정
- 현실의 문제
- ↓
- Data Collection (데이터 수집)
- ↓
- Data Cleaning (데이터 정제)
- ↓
- Data Analysis (데이터 분석)
- ↓
- Data Visualization (데이터 시각화)
- ↓
- Insight (인사이트)
- ↓
- Decision (의사결정)

02. 데이터 분석 과정에서 Python은 어디에 있는가?

2) 실생활 예시

- 온라인 쇼핑몰이라면 : “최근 매출이 왜 감소했을까?”
- Python으로 주문 데이터 10만 건 분석
- → 신규 고객은 그대로인데 기존 고객의 재구매율이 감소했다. 라는 사실을 발견할 수 있음
- Python은 단순한 개발자용 프로그래밍 언어가 아니라 **데이터를 다루기 위한 도구**

03. Excel이 있는데 왜 Python을 배울까?

구분	Excel	Python
Full Name	Microsoft Excel	Python Programming Language
주요 방식	GUI 중심	Code 중심
데이터 규모	소~중규모에 편리	대규모 처리 가능
반복 작업	수작업 발생 가능	자동화 가능
분석 재현	작업 과정 추적 어려움	Code로 재현 가능
웹 데이터	제한적	수집 가능
AI/ML	제한적	강력
입문 난이도	낮음	상대적으로 높음

03. Excel이 있는데 왜 Python을 배울까?

1) GUI = Graphical User Interface

- 그래픽 요소를 이용하여 컴퓨터를 조작하는 사용자 인터페이스
- Excel에서 셀을 클릭하고 메뉴를 누르는 것이 GUI

2) Code

- 컴퓨터에게 수행할 작업을 프로그래밍 언어로 작성한 명령

sales.mean()

- “sales의 평균을 계산해라.”

3) 핵심

- Excel을 버리고 Python을 사용하는 것이 아님!
- 실무에서는 오히려 Excel + SQL + Python + BI 처럼 함께 사용하는 경우가 많음

04. Programming이란 무엇인가?

1) Programming = 프로그래밍

- 사전적 의미 : 컴퓨터가 특정 작업을 수행하도록 명령의 순서와 규칙을 작성하는 행위

2) Program = 프로그램

- 컴퓨터가 특정 목적을 수행하기 위해 실행하는 명령들의 집합

개념	의미	비유
Program	완성된 명령 집합	요리 레시피
Programming	명령을 만드는 행위	레시피 작성
Programmer	프로그램 작성자	레시피 작성자
Computer	프로그램 실행	요리사

04. Programming이란 무엇인가?

2) 실생활 예

- 라면 끓이기 :

1. 물 550ml 준비
2. 물을 끓인다.
3. 면을 넣는다.
4. 스프를 넣는다.
5. 4분간 끓인다.

- 이것도 일종의 Program이라고 생각할 수 있음

05. Algorithm — 프로그램보다 먼저 존재하는 것

1) Algorithm = 알고리즘

- 어떤 문제를 해결하기 위한 명확하고 유한한 절차 또는 단계의 집합
- 어원은 9세기 페르시아 수학자 Muhammad ibn Musa al-Khwarizmi(무함마드 이븐 무사 알콰리즈미)의
- 라틴어식 이름에서 유래한 것으로 알려져 있음
- 인물: Al-Khwarizmi
- 활동 시기: 약 9세기
- 분야: 수학·천문학
- 역사적 의미: 계산 절차와 대수학 발전에 큰 영향

2) 프로그램과 차이

Algorithm	Program
문제 해결 절차	컴퓨터가 실행할 명령
언어 독립적	특정 언어로 구현
아이디어	구현물

05. Algorithm — 프로그램보다 먼저 존재하는 것

2) 실생활 예

- “학생 30명 중 가장 높은 점수를 찾는다.”

첫 학생의 점수를 기준으로 저장

↓

다음 학생과 비교

↓

더 높으면 교체

↓

마지막 학생까지 반복

- 위 과정은 알고리즘 | Python으로 구현하면 프로그램이 됨

Computer는 **내가 원하는 것**을
실행하는 것이 아니라
내가 명령한 것을 실행한다

06. 컴퓨터는 생각하지 않는다

1) 실생활 예

- 예를 들어 사람이 “점수가 높은 학생을 찾아줘.” 라고 하면 이해함
- 컴퓨터에게는 무엇이 높은 것인지, 어떤 데이터를 학생이라고 할 것인지, 어떤 값이 점수인지 알려줘야함

```
if score >= 90:
    print("A")
```

2) if = 만약 ~라면

- 조건에 따라 다른 명령을 실행하기 위한 Conditional Statement(조건문)
- 프로그래밍에서 중요한 능력은 “큰 문제를 컴퓨터가 이해할 수 있는 작은 문제로 분해하는 것”





07. Python이란?

1) Python Programming Language

- 읽기 쉽고 간결한 문법을 목표로 설계된 범용 고수준 프로그래밍 언어

2) 탄생 배경

항목	내용
창시자	Guido van Rossum
개발 시작	1989년
최초 공개	1991년
목적	읽기 쉽고 생산성이 높은 언어
이름 유래	뱀이 아니라 코미디 프로그램 Monty Python
현재 활용	데이터 분석, AI, 웹, 자동화 등

Python의 가장 중요한 철학

Readability (가독성)

= 사람이 읽기 쉬운 코드

08. Python은 어디에서 사용되는가?

분야	활용	대표 예
Data Analysis	데이터 분석	매출 분석
Data Visualization	데이터 시각화	차트 생성
Web Development	웹 개발	API 서버
Automation	자동화	Excel 자동 생성
Web Crawling	웹 데이터 수집	리뷰 수집
AI	인공지능	생성형 AI
ML	머신러닝	고객 이탈 예측

08. Python은 어디에서 사용되는가?

1) AI = Artificial Intelligence

- 인간의 지능적 행동을 컴퓨터 시스템으로 구현하는 기술 및 연구 분야

2) ML = Machine Learning

- 데이터에서 패턴을 학습하여 예측이나 판단을 수행하도록 하는 인공지능의 한 분야

Python → Data → Marketing → AI

컴퓨터 내부에서는 결국
Binary Number(이진수)를 사용

09. Programming Language는 왜 필요한가?

1) Binary = 이진법

- 0과 1 두 개의 숫자를 이용하는 수 체계
- 사람이 컴퓨터에게 직접 01001000 01100101 ... 처럼 명령하는 것은 어려움

2) 그래서 사람과 컴퓨터 사이에 Programming Language가 존재

- Human (사람)
- ↓ Python
- Programming Language (프로그래밍 언어)
- ↓
- Computer (컴퓨터)

Python은 사람이 **컴퓨터에게**
명령을 전달하기 위한 언어

10. Python은 어떻게 실행될까?

Python Code
소스 코드

↓

Python Interpreter

↓

Computer

↓

Result
결과

10. Python은 어떻게 실행될까?

1) Interpreter = 인터프리터

- 작성된 프로그램을 읽고 실행하는 소프트웨어

print("Hello Python")

- Python Interpreter가 이를 읽고 실행하여

Hello Python

- 이라는 결과를 출력

구분	Interpreter	Compiler
한국어	인터프리터	컴파일러
기본 개념	코드를 실행하며 처리	전체 프로그램을 번역
학습 비유	통역	번역
대표적 설명 사례	Python	C/C++

11. 이제 데이터를 컴퓨터에 넣어보자 — Variable

1) Variable = 변수

- 사전적 의미 : 프로그램에서 데이터를 저장하고 참조하기 위해 이름을 붙인 공간 또는 식별자

coffee_price = 4500

- 의미는 “coffee_price라는 이름으로 4500이라는 값을 사용할 수 있게 한다.” 정도로 이해하면 됨

2) 실생활 비유

- 냉장고의 수납통에 [우유] 라고 라벨을 붙이는 것과 비슷
- 프로그래밍에서는
- coffee_price → 4500 customer_name → "홍길동" sales → 120000
- 처럼 데이터에 이름을 붙임

12. 모든 데이터는 같은 종류가 아니다 — Data Type

1) Data Type = 자료형

- 데이터의 종류와 해당 데이터에 수행할 수 있는 연산을 정의하는 분류
- age = 25
- name = "Kim"
- height = 175.5
- is_member = True

Python	Full Name	의미	예
int	Integer	정수	25
float	Floating-point Number	실수	175.5
str	String	문자열	"Kim"
bool	Boolean	논리값	True

12. 모든 데이터는 같은 종류가 아니다 — Data Type

2) Boolean = 불리언 / 논리형

- 19세기 영국 수학자 George Boole(조지 불)의 이름에서 유래
- 인물: George Boole
- 주요 저서: The Laws of Thought
- 발표: 1854년
- 목적: 논리적 명제를 수학적으로 표현
- 현대 컴퓨팅 영향: True / False 논리 연산의 이론적 기반

is_member = True

- “이 고객은 회원인가?” → Yes / No

13. 데이터가 여러 개라면? — Data Structure

1) Data Structure = 자료구조

- 여러 데이터를 효율적으로 저장하고 접근·관리하기 위한 구조

2) Python 첫 단계에서 만나게 될 자료구조

자료구조	Full Name	특징	실생활 비유
String	String	문자들의 집합	고객 이름
List	List	순서 있는 데이터	쇼핑 목록
Tuple	Tuple	변경하기 어려운 순서 데이터	좌표
Set	Set	중복 없는 데이터	회원 ID 목록
Dictionary	Dictionary	Key-Value	회원정보

13. 데이터가 여러 개라면? — Data Structure

1) Key = 키

- 데이터를 식별하기 위한 이름

2) Value = 값

- 해당 Key에 연결된 실제 데이터

```
customer = {
   "name": "Kim",
   "age": 30,
   "grade": "VIP"
}
```

3) 실생활의 사전

- 단어(Key) → 뜻(Value)

14. 변수 → 자료형 → 자료구조의 관계

Data
↓
Value
↓
Data Type
↓
Variable
↓
Data Structure

Python을 잘한다 ≠ 문법을 많이 외운다

15. 오늘부터 가져야 할 Programming 사고방식

Problem (문제)
↓
Decomposition (문제 분해)
↓
Data (필요한 데이터)
↓
Logic (처리 방법)
↓
Code (Python 구현)
↓
Result (결과)
↓
Insight (의미 해석)

02. Python 프로그램의 흐름을 제어하는 방법

01. 어제의 Python은 아직 계산기에 가깝다

```
price = 5000
quantity = 3
discount_rate = 0.1

total = price * quantity
discount_price = total * (1 - discount_rate)

print(discount_price)
```

1) 실제 쇼핑몰이라면?

- 회원이면 10% 할인
  비회원이면 할인 없음
  구매금액이 50,000원 이상이면 무료배송
  장바구니에 상품이 있는 동안 반복해서 금액 계산

2) Decision + Repetition

- 프로그램에는 두 가지 능력이 필요 : Decision + Repetition
  - Decision = 판단 | Repetition = 반복

02. Program은 항상 위에서 아래로만 움직일까?

1) Sequential Execution = 순차 실행

- 명령문을 작성된 순서대로 하나씩 실행하는 방식

이름 입력
↓
가격 입력
↓
수량 입력
↓
총액 계산
↓
결과 출력

price = 5000
quantity = 3
total = price * quantity
print(total)

2) 세상의 모든 문제가 이렇게 단순하지 않음

- 프로그램의 실행 방향을 바꿀 필요가 생김

로그인 성공?
├ Yes → 메인페이지
└ No → 오류 메시지

03. Control Flow — 프로그램의 교통신호

1) Control Flow = 제어 흐름

- 프로그램에서 명령문이 실행되는 순서와 경로

2) 프로그램의 흐름은 크게 세 가지로 생각

구조	English	의미	실생활
순차	Sequence	순서대로 실행	요리 레시피
선택	Selection	조건에 따라 선택	신호등
반복	Iteration	작업 반복	매일 출석체크

Sequence
↓
Selection (if)
↓
Iteration (for / while)

04. Structured Programming의 등장

1) Structured Programming = 구조적 프로그래밍

- 프로그램을 순차·선택·반복 등의 명확한 제어구조를 중심으로 구성하는 프로그래밍 방식
- 1960~70년대 소프트웨어가 복잡해지면서 프로그램의 흐름을 체계적으로 관리할 필요성이 커짐



2) Edsger W. Dijkstra

- 네덜란드 컴퓨터 과학자
- 1930~2002
- 1968년 논문/서신 “Go To Statement Considered Harmful”로 잘 알려짐
- 복잡한 goto 중심 프로그램보다 명확한 구조를 가진 프로그램을 강조

복잡한 프로그램도 결국
순차 + 선택 + 반복의
조합으로 생각

05. 첫 번째 능력 — Python에게 판단시키기

1) 상황을 하나 제시

- 쇼핑몰에서 50,000원 이상 구매한 고객에게 무료배송을 제공
- 사람이라면 바로 판단 : 구매금액 >= 50,000원 ?
- Python도 동일

```
total_price = 70000


if total_price >= 50000:
    print("무료배송")
```

2) Conditional Statement = 조건문

- 주어진 조건의 참·거짓에 따라 서로 다른 명령을 실행하도록 하는 문장
- Python에서는 대표적으로 if를 사용

06. if의 작동 원리

1) if

- 영어 : if = 만약 ~라면

if 조건:
** 실행할 코드**

2) 예시

age = 25

if age >= 20:
** print("성인입니다.")**

age >= 20
↓
True ?
↓
YES
↓
"성인입니다."

- 조건식의 결과는 결국 : True | False
- Boolean과 if가 연결

07. Boolean → Comparison → if

1) Boolean = 불리언

- True 또는 False의 두 가지 논리 상태를 표현하는 자료형

2) Comparison Operator = 비교 연산자

- 두 값을 비교하여 True 또는 False를 반환하는 연산자

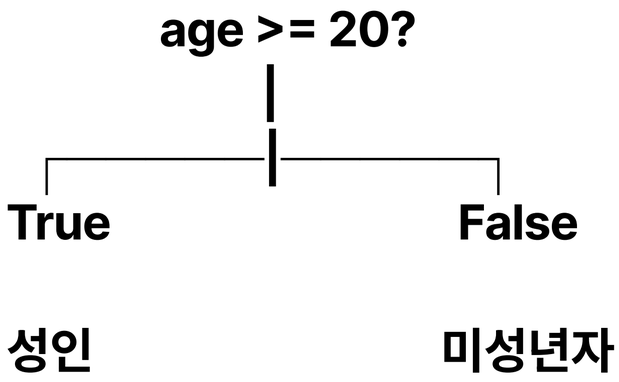
연산자	의미	예
==	같다	age == 20
!=	다르다	age != 20
>	크다	age > 20
<	작다	age < 20
>=	크거나 같다	age >= 20
<=	작거나 같다	age <= 20

08. if / else — 두 갈래 길

1) else = 그렇지 않으면

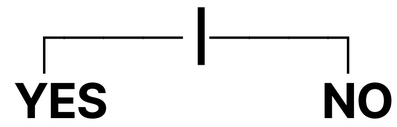
```
age = 18

if age >= 20:
    print("성인")
else:
    print("미성년자")
```



2) 실생활 예

- 우리가 매일 사용하는 프로그램도 이런 구조



로그인 오류 메시지

09. elif — 선택지가 여러 개라면?

1) elif = else if

- 앞선 조건이 False일 때 추가적인 조건을 검사하는 Python 키워드

```
score = 85

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
else:
    print("D")
```

```
purchase = 150000

if purchase >= 200000:
    print("VIP")
elif purchase >= 100000:
    print("Gold")
else:
    print("Normal")
```

2) 마케팅 데이터로 바꿔보면

- 나중에 배우게 될 Customer Segmentation(고객 세분화)의 아주 작은 출발점

10. Indentation — Python에서는 들여쓰기가 문법이다

1) Indentation = 들여쓰기

- 코드의 시작 위치를 일정한 간격만큼 오른쪽으로 이동시키는 것

```
if age >= 20:                    if age >= 20:
    print("성인")                    print("성인")
    print("입장 가능")

                                 print("프로그램 종료")
```

- 왼쪽 : 두 print() 모두 조건문에 포함
- 오른쪽 : 마지막 print()는 조건과 관계없이 실행

2) Python Style Guide

- Python에는 공식적인 코딩 스타일 가이드인 PEP 8 존재
- PEP = Python Enhancement Proposal : Python의 기능, 설계, 개발 관례 등을 제안·문서화하는 체계

3) PEP 8 = Style Guide for Python Code

- 주요 작성자: Guido van Rossum, Barry Warsaw, Alyssa Coghlan
- 최초 작성: 2001년
- 목적: Python 코드의 일관성과 가독성 향상
- 일반적인 들여쓰기 권장: Space 4칸

11. 두 번째 능력 — 반복되는 일을 자동화하기

1) 질문

- 학생 5명의 이름을 출력한다면 :

```
print("Kim")
print("Lee")
print("Park")
print("Choi")
print("Jung")
```

- 5명은 가능 / 그런데 고객이 100,000명이라면?
- print()를 10만 번 작성할 수는 없음

2) Loop = 반복문

- 특정 코드 블록을 여러 번 실행하도록 하는 프로그래밍 구조
- Python의 대표적인 반복문 : for, while

12. for — 데이터가 여러 개라면?

1) for Statement = for 반복문

- 순회 가능한 데이터의 각 요소를 차례대로 가져오면서 동일한 작업을 반복하는 구조

```
customers = ["Kim", "Lee", "Park"]

for customer in customers:
    print(customer)
```

```
Kim
Lee
Park
```

- List가 반복문과 만나기 시작

13. Iteration — 하나씩 꺼내서 처리한다

1) Iteration = 반복 / 순회

- 일련의 데이터나 명령을 차례대로 반복 처리하는 과정

2) 출석 확인 / 데이터 분석 예시

학생 명단
↓
첫 번째 학생 → 출석?
↓
두 번째 학생 → 출석?
↓
세 번째 학생 → 출석?
↓
...

100,000개 주문
↓
각 주문의 매출 계산
↓
각 고객의 구매액 계산
↓
각 리뷰의 평점 확인

- 그래서 반복문은 데이터 분석에서 매우 중요

14. range() — 횟수를 정해서 반복한다

1) range = 범위

- Python의 range()는 일정한 정수 범위를 표현하는 객체를 생성

```
for i in range(5):
    print(i)
```

```
0
1
2
3
4
```

- 초보자들이 가장 많이 묻는 질문 : 왜 1~5가 아니라 0~4인가?
- Python을 포함한 많은 프로그래밍 환경에서 위치를 나타내는 Index(인덱스)가
- 0부터 시작하는 경우가 많음

2) Index = 색인 / 위치 번호

- 데이터 집합에서 특정 요소의 위치를 식별하기 위한 번호

데이터	Kim	Lee	Park
Index	0	1	2

15. while — 언제 끝날지 모른다면?

1) while = ~하는 동안

- 주어진 조건이 True인 동안 코드 블록을 반복 실행하는 구조

```
count = 1

while count <= 5:
    print(count)
    count = count + 1
```

2) for와 while 비교

구분	for	while
기본 의미	데이터 순회	조건 기반 반복
적합한 상황	반복 대상/범위가 명확	종료 시점이 조건에 따라 결정
예	학생 30명 확인	비밀번호가 맞을 때까지 입력
주의	비교적 안전	Infinite Loop 가능

15. while — 언제 끝날지 모른다면?

1) Infinite Loop = 무한 반복

- 종료 조건이 충족되지 않아 프로그램이 계속 반복되는 현상

while True:
** print("계속 실행")**

- while에서는 **언제 반복을 끝낼 것인가?**가 매우 중요

16. for vs while — 언제 무엇을 사용할까?

상황	추천	이유
학생 30명 출력	for	데이터 수가 정해짐
상품 100개 가격 계산	for	상품을 하나씩 순회
1~10 출력	for	반복 범위 명확
비밀번호 맞을 때까지 입력	while	성공 시점 불확실
게임 HP가 0이 될 때까지	while	조건에 의해 종료
파일 목록 순회	for	데이터 집합 순회

1) 기억법

- for = 무엇을 하나씩 처리할 것인가?
- while = 언제까지 계속할 것인가?

17. 조건문 + 반복문이 만나면 프로그램다워진다

1) 쇼핑몰 주문금액 :

orders = [30000, 70000, 120000, 45000]

- 50,000원 이상 주문만 찾고 싶은 경우

```
for order in orders:

    if order >= 50000:
        print(order)
```

- 결과 : 70000 120000

2) Data Filtering = 데이터 필터링

- 주어진 조건을 만족하는 데이터만 선택하는 작업

orders
↓
for
하나씩 가져오기
↓
if
조건 판단
↓
조건을 만족한 데이터
↓
Output

03. Python 코드를 기능으로 묶는 방법

01. 어제까지 Python이 할 수 있게 된 것

```
price = 5000
quantity = 3

total = price * quantity
discount = total * 0.1
final_price = total - discount
```

이 계산을 **주문 100건마다**
계속 작성해야 한다면?
같은 코드를 계속 복사해야 할까?

02. Reusability — 프로그램은 반복 작성하지 않는다

1) Reusability = 재사용성

- 이미 작성한 코드나 기능을 다시 사용할 수 있는 성질

2) 실생활 예시

- 전자레인지에는 [데우기 해동 자동조리] 라는 기능이 미리 정의
- 사용자는 내부 회로를 매번 새로 만들지 않고 버튼만 클릭

3) 프로그래밍 예시

- 복잡한 코드
- ↓
- Function으로 묶기
- ↓
- 필요할 때 호출

03. Function이란 무엇인가?

1) Function = 함수

- 특정 작업을 수행하도록 묶어 놓은 재사용 가능한 코드 블록

def say_hello():
    print("Hello")

say_hello()

Hello

Function Definition
함수 정의
↓
Function Call
함수 호출
↓
Execution
실행

04. 함수라는 개념은 어디에서 왔을까?

1) Mathematical Function = 수학적 함수

- 하나의 입력값에 대해 하나의 출력값을 대응시키는 관계

f(x) = x × 2　　x = 5　　10

2) Leonhard Euler(레온하르트 오일러)

항목	내용
인물	Leonhard Euler
활동 시기	18세기
생몰	1707~1783
기여	함수 표기와 해석학 발전
대표적 표기	f(x)
목적	변수 간 관계를 수학적으로 표현

05. Python Function의 기본 구조

1) def = define

- 함수를 정의할 때 사용하는 Python Keyword

def function_name():
** 실행할 코드**

def greeting():
** print("안녕하세요")**

greeting()

2) Function Definition = 함수 정의

- 함수가 어떤 작업을 수행할지 작성하는 것

3) Function Call = 함수 호출

- 정의된 함수를 실제로 실행하는 것

06. 함수도 이름이 중요하다

1) 좋지 않은 함수 :

- 기능을 알기 어려움

def test():

...

2) 좋은 함수 :

def calculate_sales():

...

3) Naming Convention = 명명 규칙

- 변수·함수 등의 이름을 일관된 방식으로 작성하는 규칙
- Python에서는 함수 이름 역시 일반적으로 snake_case 를 사용

4) 예시

- calculate_total_price()
- get_customer_name()
- save_excel_file()

07. 함수가 없다면 어떻게 될까?

1) 상품 가격 계산 :

price1 = 5000
quantity1 = 3
total1 = price1 * quantity1

price2 = 12000
quantity2 = 2
total2 = price2 * quantity2

price3 = 8000
quantity3 = 5
total3 = price3 * quantity3

2) 반복되는 부분 :

가격 × 수량

07. 함수가 없다면 어떻게 될까?

3) 함수로 만들면 :

```
def calculate_sales(price, quantity):
    return price * quantity
```

4) 기대효과 :

```
calculate_sales(5000, 3)
calculate_sales(12000, 2)
calculate_sales(8000, 5)
```

08. Parameter — 함수에게 데이터를 전달한다

1) Parameter = 매개변수

- 함수를 정의할 때 외부에서 전달받을 값을 대신하여 사용하는 변수

def calculate_sales(price, quantity):
** return price * quantity**

- 여기에서 price quantity가 Parameter

2) 실생활 비유

- 커피 머신을 생각하면 원두 물 컵 크기가 입력값
- 커피 머신은 입력값에 따라 결과를 만듭니다. = Function도 같습니다.

09. Argument — 실제로 전달하는 값

1) Argument = 인자

- 함수를 호출할 때 실제로 전달하는 값

calculate_sales(5000, 3)

- 여기에서 5000 3이 Argument입니다.

구분	Parameter	Argument
한국어	매개변수	인자
위치	함수 정의	함수 호출
의미	값을 받을 이름	실제 전달되는 값
예	price	5000

09. Argument — 실제로 전달하는 값

2) 코드 :

```
def calculate_sales(price):
    return price
```

3) 호출 :

```
calculate_sales(5000)
```

4) 정리하면 :

price → Parameter
5000 → Argument

10. Return — 함수의 결과를 돌려준다

1) Return = 반환

- 함수가 처리한 결과값을 호출한 위치로 돌려주는 것

2) 예시 :

```
def calculate_sales(price, quantity):
    return price * quantity
```

3) 호출 :

```
result = calculate_sales(5000, 3)
```

4) 결과 :

```
15000
```

10. Return — 함수의 결과를 돌려준다

구분	print()	return
목적	화면 출력	결과 반환
이후 재사용	어려움	가능
함수 종료	아님	일반적으로 함수 종료
데이터 저장	직접적이지 않음	변수에 저장 가능

```
def test():
    print(10)
```

```
def test():
    return 10
```

11. Input → Process → Output으로 함수 이해하기

1) 컴퓨터의 기본 구조

Input
↓
Process
↓
Output

2) 예시

```
def calculate_discount(price, rate):
    result = price * (1 - rate)
    return result
```

11. Input → Process → Output으로 함수 이해하기

3) 분해하면 :

단계	코드	의미
Input	price, rate	입력
Process	price * (1-rate)	처리
Output	return result	출력

4) 실생활 예 :

가격 50,000원
할인율 20%
↓
할인 계산
↓
40,000원

12. Built-in Function — Python이 미리 만들어 준 함수

1) Built-in Function = 내장 함수

- Python이 기본적으로 제공하여 별도의 정의 없이 바로 사용할 수 있는 함수

함수	Full Name / 의미	예
print()	출력	print("Hello")
input()	입력	input("이름: ")
len()	Length	데이터 길이
type()	Type	자료형 확인
int()	Integer	정수 변환
str()	String	문자열 변환
float()	Floating-point	실수 변환

13. User-defined Function — 우리가 직접 만드는 함수

1) User-defined Function = 사용자 정의 함수

- 프로그래머가 특정 목적을 위해 직접 정의한 함수

```
def calculate_average(a, b, c):
    return (a + b + c) / 3
```

2) Built-in vs User-defined

구분	Built-in Function	User-defined Function
제작자	Python	개발자
사용	바로 사용	먼저 정의
예	len()	calculate_sales()
목적	공통 기능	특정 업무

14. Module — 함수가 많아지면 어떻게 할까?

1) Module = 모듈

- 관련된 함수·변수·클래스 등을 하나의 Python 파일로 묶어 놓은 단위

2) 예를 들어 :

- calculator.py 안에 아래와 같은 함수들이 들어갈 수 있습니다.

3) 안에 :

def add():

...

def subtract():

...

def multiply():

...

14. Module — 함수가 많아지면 어떻게 할까?

4) 실생활 비유

- 회사 조직처럼 생각하면 쉬움
- 관련 업무를 한 곳에 묶어 관리가능

회사

— 마케팅팀

— 개발팀

└— 디자인팀

15. import — 다른 Module을 가져온다

1) import = 가져오기

- 다른 Module의 기능을 현재 프로그램에서 사용할 수 있도록 불러오는 Python Keyword

import math

math.sqrt(16)

4.0

2) math = Mathematical Functions Module

- 수학 관련 기능을 제공하는 Python 표준 모듈

import random

3) Random Module = 난수 관련 모듈

- 무작위 값을 생성하는 기능을 제공하는 Python 표준 모듈

random.randint(1, 10)

16. Module → Package → Library는 무엇이 다른가?

개념	English	기본 의미	비유
Module	Module	하나의 Python 파일	문서 1개
Package	Package	여러 Module의 묶음	폴더
Library	Library	특정 목적의 기능 모음	도서관

1) Package = 패키지

- 서로 관련된 Python Module들을 구조적으로 묶어 관리하는 단위

2) Library = 라이브러리

- 특정 작업을 쉽게 수행할 수 있도록 미리 만들어 놓은 기능들의 집합
- 대표적인 데이터 분석 Library:
  - NumPy
  - Pandas
  - Matplotlib

17. 왜 함수와 모듈이 데이터 분석에서 중요할까?

1) 예시

- 매번 동일한 코드를 복사하면 오류가 늘어남

파일 불러오기
↓
결측값 제거
↓
날짜 변환
↓
매출 계산
↓
결과 저장

- 함수로 만들면 재사용 가능

def clean_sales_data(data):
...

def calculate_total_sales(data):
...

17. 왜 함수와 모듈이 데이터 분석에서 중요할까?

2) Data Pipeline = 데이터 파이프라인

- 데이터가 수집·정제·변환·분석되는 일련의 처리 흐름
- 함수는 나중에 이 데이터 파이프라인을 만드는 기본 단위가 됨

Raw Data
↓
Cleaning Function
↓
Analysis Function
↓
Visualization Function
↓
Report